

1. Aviso de autoria e escopo do projeto

É importante ressaltar que este trabalho tem como foco exclusivo a aplicação de práticas de DevOps (containerização, orquestração com Kubernetes e Helm).

A aplicação utilizada para a construção desta infraestrutura é um projeto Open Source desenvolvido por Adebayo Adesina, disponível no repositório original:

https://github.com/adebayoAdesina/Hotel_Booking_Full_stack_Application.

Observação: O README.md original do repositório foi mantido inalterado. Todas as informações referentes à entrega dessa disciplina (roteiro, arquitetura e infraestrutura) estão contidas unicamente nesse documento PDF.

2. Aplicação e seus componentes

A aplicação "lakeside_hotel" segue uma arquitetura multicamadas, containerizada e agora disponibilizada via kubernetes. O ecossistema é composto por três componentes principais:

1. Database (lakeside-db): Container baseado na imagem oficial do mysql:8.0, responsável pela persistência dos dados.
2. Backend (api-backend): Container executando a API RESTful desenvolvida em Spring Boot, que realiza a comunicação via JDBC com o banco de dados e expõe as regras de negócio.
3. Frontend (frontend): Container executando a aplicação SPA em React JS, provida por um servidor web (Nginx), responsável pela interface com o usuário.

3. Evolução da arquitetura

Para garantir a plena execução da aplicação e resolver os problemas apontados na validação via Docker Compose, as seguintes melhorias arquiteturais foram implementadas na camada de infraestrutura do Kubernetes:

3.1. Correção de comunicação entre Frontend e Backend

Para a implantação no Kubernetes, o problema de comunicação foi resolvido implementando um proxy reverso no nginx do frontend. Agora, o Axios faz requisições utilizando rotas relativas (/api/v1). O Nginx intercepta essas requisições e as encaminha internamente para o serviço do Backend no cluster. Isso garante uma comunicação robusta, sem bloqueios de CORS e sem expor portas desnecessárias ao ambiente externo.

3.2. Implementação de health checks

Para assegurar que a aplicação só receba tráfego quando estiver plenamente executável, foram adicionadas probes de liveness e readiness no Helm Chart:

1. Database: Utiliza o mysqladmin ping para garantir que o banco está pronto para conexões.
2. Backend: Monitora a porta TCP do Spring Boot com um initialDelay estendido, dando tempo suficiente para a JVM compilar e iniciar antes do tráfego ser roteado.
3. Frontend: Realiza requisições HTTP locais no Nginx para confirmar a saúde da interface gráfica.

4. Artefatos kubernetes utilizados

Toda a implantação foi automatizada utilizando o Helm. Foi criado um Helm Chart personalizado (lakeside-helm-chart) que centraliza a configuração dos seguintes artefatos:

1. Deployment: Responsável por gerenciar os pods da aplicação (Frontend, Backend e Database), garantindo auto-healing e a injeção correta das imagens previamente construídas.
2. Service: Configurados como ClusterIP, criam pontos fixos de rede interna. O serviço database, por exemplo, permite que o Spring Boot se conecte ao banco usando a variável de ambiente DB_HOST.
3. Secret: Utilizado para isolar e proteger informações sensíveis. A senha do banco de dados (MYSQL_ROOT_PASSWORD) foi removida do código e parametrizada como um objeto opaco, sendo injetada apenas em tempo de execução.
4. Ingress: Atua como o ponto de entrada do cluster. Ele captura as requisições HTTP destinadas à URL k8s.local e faz o roteamento direto para o service do Frontend na porta 80.
5. Obs: A automação do build das imagens e carregamento no Minikube foi encapsulada no script build.sh, cumprindo o requisito alternativo ao Docker Hub.

5. Roteiro de testes

Para avaliar o correto funcionamento da aplicação e de sua infraestrutura, siga os passos abaixo.

5.1. Roteiro de build

1. Adicione a seguinte linha ao arquivo “/etc/hosts” do sistema operacional: 127.0.0.1 k8s.local (ou o IP gerado pelo comando minikube ip no Linux).
2. Na raiz do projeto, garanta permissão de execução e rode o script de automação: “chmod +x build.sh ./build.sh” (Este script iniciará o Minikube, fará o build das três imagens Docker, carregará as imagens no cluster via minikube image load e instalará o Helm Chart).
3. Em uma nova janela do terminal, inicie o túnel de rede do Minikube (obrigatório para habilitar o Ingress localmente no Linux): “minikube tunnel”
4. Verifique a saúde dos containers com o comando kubectl get pods.
 - a. Importante: O pod do backend (Spring Boot) costuma demorar cerca de 2 minutos a mais para rodar e ficar pronto em relação aos outros pods. Realize os testes apenas quando todos os pods estiverem com o Status Running e Ready 1/1.

5.2. Caminho de testes

A aplicação original apresenta certas inconsistências em nível de código (não relacionadas à infraestrutura). Para validar a comunicação plena do ecossistema (Frontend -> Proxy Reverso -> Backend -> Banco de Dados), o fluxo de "adicionar quarto" foi mapeado como o teste principal. Com a aplicação aberta no k8s.local, siga os seguintes passos:

1. No menu superior do sistema, navegue pela rota: Admin -> Manage Rooms.
2. Preencha o formulário seguindo os dados abaixo:
 - a. Room Type: Selecione a opção vazia (" ").
 - i. Observação: Na primeira vez cadastrando um quarto, às vezes nem é possível acessar essa opção de seleção. Caso seja possível alterar o room type, certifique-se de selecionar a opção " ", pois a feature de adição de novos tipos de quarto está parcialmente implementada no sistema original, podendo ocasionar erros, caso não seja possível alterar o tipo de quarto, apenas ignorar e seguir para os passos abaixo.
 - b. Room Price: Insira um valor numérico arbitrário.
 - c. Room Photo: Faça o upload de uma imagem genérica qualquer.
3. Clique no botão Save Room. Uma mensagem de sucesso será exibida.
4. Para confirmar a persistência no Banco de Dados, acesse a aba Browse all rooms e verifique se o quarto criado está listado com a foto enviada.